

Leveraging Open Source LLMs for Software Engineering Education and Training

Juanan Pereira, Juan-Miguel López, Xabier Garmendia, Maider Azanza

Dept. of Computer Languages and Systems

University of the Basque Country, UPV/EHU

Donostia, Spain

{juanan.pereira, juanmiguel.lopez, xabier.garmendiad, maider.azanza}@ehu.eus

Abstract—Generative AI, particularly Large Language Models (LLMs), presents innovative opportunities to enhance software engineering education. Open source LLMs such as LLaMA and Mistral leverage the potential of generative AI offering distinct advantages over proprietary options including transparency, customizability, collaboration, and cost savings. This paper develops a catalog of LLM prompt examples tailored for software engineering training, mapped to knowledge areas from the Software Engineering Body of Knowledge (SWEBoK) framework. Example prompts demonstrate LLMs' capabilities in eliciting requirements, diagram generation, API simulation, effort estimation through role-playing, and other areas. The methodology involves evaluating prompt responses from ChatGPT, Mistral, and LLaMA on representative tasks. Quantitative and qualitative analysis assesses quality, usefulness, and correctness. Findings show ChatGPT and Mistral outperforming LLaMA overall, but no model perfectly executes complex interactions. We examine implications and challenges of integrating open source LLMs into classrooms, emphasizing the need for oversight, verification, and prompt design aligned with pedagogical objectives.

Index Terms—Software Engineering Education, Open Source AI Models, Large Language Models, Prompt Engineering

I. INTRODUCTION

Software Engineering (SE) is a dynamic and intricate discipline that continually evolves, requiring constant learning and adaptation to remain at the forefront of technological advancements. The application of Generative Artificial Intelligence (GenAI) through its application in Large Language Models (LLMs) to SE tasks presents opportunities through the automation of activities like coding, design, and documentation, but also risks around reliability and effectiveness that have broader implications needing consideration [1].

The adaptability and potential of LLMs in general, lie in their ability to encourage motivation, engagement, and learner autonomy by providing personalized, interactive support, thereby enhancing the learning experience [2].

LLM-assisted software engineering promises to support developers with expert knowledge over the whole software lifecycle in a conversational way [3].

However, it is important to note that the quality of LLM output is currently heavily dependent on the quality and scope of their training data, which in the case of software engineering area is still under development.

In the context of SE, LLMs are increasingly utilized for tasks such as code explanation, [4], requirements information

retrieval [5], identifying failure-inducing test cases [6], program repair [7] and vulnerability detection [8].

For instance, it has been demonstrated that prompt design techniques for SE in the form of patterns can be applied to improve requirements elicitation, rapid prototyping, code quality, refactoring, and system design [9]. Moreover, automatic test generation from general bug reports has also been explored using LLMs [10].

Open research challenges around LLMs and SE lie in ensuring the reliability, efficiency and effectiveness of LLMs for automating activities like coding, design, requirements specification, repair, refactoring, optimization, documentation and analytics, given LLMs' tendency to hallucinate solutions [11].

With regard to the educational context, introducing SE concepts to students presents a notable challenge for educators, as highlighted in various studies [12], [13]. The field is in a perpetual state of flux, characterized by the frequent emergence of new programming languages, tools, and methodologies [14]. This constant evolution, coupled with the diverse backgrounds and experience levels of students, makes traditional teaching methods like lectures and lab exercises less effective [15].

In response, there has been a noticeable transition over the past few years from teacher-focused learning methods to student-centered ones [16]. This shift emphasizes the need for innovative technology to facilitate an effective learning process [17].

Other problems encountered in teaching SE were related to the course contents [18]. In this context, the Software Engineering Body of Knowledge (SWEBoKv3) guide [19] serves as an essential framework, outlining the key knowledge areas (KAs) in SE education¹. Just as LLMs have the potential to support software engineering, they also hold significant promise for learning it, by offering innovative solutions to improve the teaching and learning experience in these KAs. GenAI, particularly through its application in LLMs such as ChatGPT, Anthropic Claude, and Google Bard, presents a novel way to interactively engage students in the software engineering curriculum. These models use advanced machine

¹It is worth noting that an updated version of the SWEBoK guide (2024 edition) will be released soon, expanding the number of KAs to 18. While this update does not directly affect the findings of this paper, it highlights the evolving nature of the field.

learning algorithms to understand and generate human-like text, providing a unique tool for educators to create dynamic and responsive teaching materials [20].

In this paper, we delve into the utilization of GenAI within the context of SE education, aligning with the KAs defined by SWEBoK. Our approach focuses on developing a set of prompt examples specifically designed for software engineering education using LLMs. We are particularly keen on exploring and evaluating the effectiveness of open-source LLMs like LLaMA or Mistral to take advantage of their transparency, customizability, collaboration, or cost savings aspects. Therefore, we compare their performance and suitability for educational purposes against a proprietary model, GPT-3.5 (the only model available for free to students at the time of this writing). By doing so, we hope to offer insights into how different LLMs can be utilized in software engineering training.

Despite the promising potential of GenAI in education, we also address the challenges and limitations inherent in its application, such as concerns over code quality, overreliance on automation, and the need for thoughtful integration within teaching strategies. By examining recent research and case studies, this paper aims to offer an overview of GenAI's role in SE education, providing a foundation for future research and application in this rapidly evolving field.

On balance, we contribute by:

- exploring the benefits of open source LLMs in SE education (Section III),
- specifying specific examples showing how to use GenAI prompts for teaching ideas in each SWEBoK KA (Table I)
- evaluating the responses from models like ChatGPT, Mistral, and LLaMA on representative tasks in software engineering (Section VI),
- discussing on the potential and challenges of integrating LLMs into software engineering education (Section VII).

This paper presents an initial exploration into the potential of open-source LLMs for software engineering education, examining their effectiveness across various SE tasks and reflecting on the benefits and challenges of integrating them into classrooms. The paper is structured as follows: Section 2 provides a background on Generative AI and open-source LLMs. Section 3 discusses the benefits of open-source LLMs in the context of SE education. Section 4 explores the use of LLMs in the software engineering classroom, and Section 5 outlines our methodology. Section 6 presents our evaluation results, and Section 7 discusses the implications and challenges of our findings. Finally, Section 8 offers concluding remarks and directions for future work in this area.

We start introducing generative AI and open source LLMs.

II. A BRIEF ON GENERATIVE AI AND OPEN SOURCE LLMs

GenAI is at the forefront of creating new, original content through artificial intelligence, utilizing machine learning algorithms to produce diverse forms of data such as text, images,

music, and videos. These models are capable of recognizing complex patterns and learning from extensive datasets. Their realistic and intricate outputs mimic human creativity, finding practical applications in various fields such as medicine [21] or software development [1]. In the realm of text generation, open-source LLMs like Mistral and LLaMA have garnered attention.

A. Mistral and MistralAI

MistralAI is an artificial intelligence company focused on developing large language models that can understand and generate natural language. Their latest model, Mistral-8x-7B, introduces a new Sparse Mixture-of-Experts (SMoE) architecture that achieves state-of-the-art performance while being more efficient.

Specifically, Mistral is composed of multiple “expert” modules that can each process a subset of the input data. A router mechanism dynamically selects which few experts process each part of the input. This means that even though Mistral has a very high overall capacity with 47 billion parameters, each part of the input only uses 13 billion active parameters.

Through this efficient architecture, Mistral matches or exceeds previous state-of-the-art models like GPT-3.5 and LLaMA 2 despite having far fewer parameters. It demonstrates strong performance across diverse Natural Language Processing (NLP) tasks including mathematics, code generation, reading comprehension, common sense reasoning, and multilingual understanding.

While Mistral-8x7B is one of the best open source LLM, its enormous size (46.7B parameters) means that the model cannot be fully loaded on a consumer GPU (e.g., a nVidia RTX 3090 GPU with 24 GB of VRAM is not enough), even when quantized to 4 bits. In the context of LLMs, quantization refers to a process that reduces the precision of the model's numerical data to make it more efficient for computation and storage. Essentially, it's a technique to compress the model size without significantly compromising its performance [22].

Even though we have used Mistral just in a SE training context, by developing Mistral and releasing both the base and fine-tuned models publicly under an open license, MistralAI aimed to push forward research and development of safe, capable AI that can serve a wide range of uses.

B. LLaMA

Meta AI's LLaMA (Large Language Model Meta AI) is a family of language models with various sizes, ranging from 7 to 70 billion parameters. Released in 2023, LLaMA models have demonstrated competitive performance on NLP benchmarks. Notably, the 13-billion parameter model outperformed the much larger GPT-3 in many respects. LLaMA's training datasets are comprehensive, including web pages, open-source code, Wikipedia, and more, ensuring a diverse knowledge base. Meta released LLaMA under a noncommercial license, significantly contributing to AI research and development.

Unlike proprietary models such as GPT, these open-source alternatives offer a unique set of advantages in terms of

privacy, cost, transparency, community-driven development, customization, avoiding vendor dependency, and educational value. The next section explores the potential of these benefits for SE education.

III. BENEFITS OF OPEN SOURCE LLMs

The integration of open-source LLMs into SE education presents a realm of untapped potential, offering distinct advantages over their proprietary counterparts. These open-source models not only democratize access to advanced AI technology but also address several concerns associated with proprietary LLMs, such as privacy, compliance with regulations like GDPR, and financial costs.

Transparency: Open-source LLMs provide a transparent and customizable platform for educators and students. Unlike proprietary models, where the internal workings are often opaque, open-source models allow SE educators to understand, modify, and adapt the algorithms to suit specific educational needs [23]. This transparency is crucial in a learning environment, as it enables educators to tailor the models to deliver more relevant and effective educational content. Furthermore, the ability to inspect and modify the codebase of open-source LLMs encourages a deeper understanding of the technology, fostering an environment of learning and experimentation that is vital in SE education [24].

Avoiding Vendor Lock-in: Using an open source LLM avoids vendor lock-in, where users are dependent on a specific provider's tools and services. This freedom allows for more flexibility in changing or upgrading systems as needed.

Privacy: Privacy and data protection are significant concerns with proprietary LLMs, especially in regions governed by strict data protection laws like the GDPR [25]. Open-source LLMs offer greater control over data, allowing institutions to host these models on their servers, thus ensuring that sensitive information, such as student data, remains within the control of the educational institution. This aspect is particularly relevant in scenarios where data privacy is paramount.

Financial costs: Economically, open-source LLMs present a more viable option for educational institutions. Proprietary models often come with licensing fees and usage costs, which can be prohibitive for many schools and universities, especially those with limited budgets. Open-source software, including Open-source LLMs, offers a cost-effective alternative to proprietary software in education. Being free to use and modify, alleviates these financial burdens, making advanced AI technology more accessible to a broader range of educational institutions [26].

In contrast, proprietary LLMs, while often more polished and user-friendly, come with the caveat of higher costs, potential privacy concerns, and a lack of transparency that can impede their full utilization in educational settings. These models, bound by company policies and profit motives, may not always align with the educational goals of SE programs.

Seminal works, such as those by White et al. [9], highlight the importance of carefully crafted prompts and strategies in leveraging LLMs effectively in SE. These models offer

innovative solutions for common challenges in SE activities. The next section explores the use of the LLMs for activities conducted during in the Software Engineering Classroom.

IV. LLMs IN THE SOFTWARE ENGINEERING CLASSROOM

In January 2023, the authors of this manuscript started teaching a new edition of the Software Engineering (SoftEng) undergraduate course with second-year students of the Computer Science Degree at the University of the Basque Country (UPV/EHU). During the course, we wanted to investigate and test the capabilities of ChatGPT for its use in the SoftEng classroom (later, when LLaMA2 and Mistral AI released their models under an Open Source license, we tested them for this paper). ChatGPT chatbot was published by OpenAI just two months before our course started, in November 2022.

The students of the SoftEng course must complete a practical project within 4 months. Starting from a statement, they perform the analysis, design, implementation and testing of a desktop application developed in Java and JavaFX. In 2023 the students, grouped in teams of 4 people, were asked to develop a Mastodon desktop client. Briefly, Mastodon can be described as a decentralized alternative to X (formerly known as Twitter). One of the assignments to accomplish by the students in the course is the requirements analysis and use case gathering: **identifying the functional and non-functional requirements to be implemented**. Given their limited experience, students often struggle with this. They face a blank sheet of paper on which they need to write a list of the functionalities the application should offer. This is where ChatGPT, Mistral, LLaMA and other LLMs can help by creating a first draft to help overcome this first stumbling block.

Brainstorming. The following prompt generated a list of functional and non-functional requirements, which served to promote a brainstorming of ideas to be developed among team members during the first iteration.

“A student has been asked to produce a list of use cases for an application that she must develop. The application is a Mastodon desktop client that must be implemented in Java and JavaFX. The student has drafted the following list of use cases:”

Diagram generation. The above requirements capture phase is usually accompanied by two UML artifacts: a domain model and a use case diagram. The analyzed chatbots (ChatGPT, LLaMA, Mixtral) do not generate graphs or diagrams directly without the help of plugins, but we can use a detour to generate UML diagrams. Just asking the chatbot to output its response in PlantUML format does the trick. PlantUML² is a tool to quickly generate UML diagrams from plain text. The following prompt allowed to generate a domain model of the entities and relationships existing in the Mastodon client project:

“Identify the entities and relationships of the mastodon client project. Give your answer in PlantUML format”

²<https://plantuml.com/>

The answer in PlantUML format can be pasted into an online PlantUML viewer (such as planttext.com) to quickly view the results.

Simulation. Among the suggested functional requirements was the Timeline View feature. The user should be able to view their home timeline, which includes the posts of the users they follow, in chronological order, within the desktop client. To implement this functionality - and others - students wanted to use the Mastodon API. However, in the early stages of the project there is no information yet on how to interact with this API. It is necessary to simulate it. This is another advantage offered by ChatGPT and similar generative AIs. A GenAI can effectively simulate an API and guide the student to make good design decisions that result in a robust, scalable and usable application. The interactive nature of working with GenAI also provides a valuable learning experience for students. See <https://sharegpt.com/c/Ik2FYBZ5> for an example where the student can easily ask the chatbot to simulate a REST API (not yet implemented) with a simple line. The following prompt gets a response from ChatGPT where it returns a JSON object that we could receive if we called a hypothetical api/timeline method of the Mastodon API: "Can you generate a draft HTTP API for the timeline view?"

Role playing estimations. Another area of a software project where students struggle due to the complexity of the topic and their inexperience is effort estimation. Again, GenAI can help to improve this skill by following the technique of role-playing estimations. With this technique we can ask the IA to ask the learner to estimate the effort required to complete certain functionalities. The IA will then respond, as a counsellor, by giving feedback on the estimate. The following prompt is an example of this technique:

"I want you to ask me about estimating 3 use cases, one at a time. For each one, I will answer you with a rough estimate in hours. Then you have to give me feedback (with your opinion on my estimate for each use case)" (Fig. 2).

This role playing usage of the AI is interesting, not only because of the immediate feedback, but also because lecturers can use it to measure the accuracy of students' relative estimates (according to the student, is task A more or less difficult to implement than task B, and why is this so?).

Based on our experience we collected a set of useful prompts - like the ones described above - and categorized them in 12 knowledge areas (KAs), following the classification defined by the SWEBOOK guideline (see Table 1). For each KA we added one or two specific examples, describing the type of activity (role playing, simulation, brainstorm, etc.) and providing a specific example of a prompt and its response in ChatGPT, Mistral8x7b and LLama2. Any interested educator can follow that table and get some actionable ideas to apply in their classroom.

V. METHODOLOGY

Following the scenario described in Section IV for the Mastodon Application Design Project, we asked GPT, LLAMA

and Mixtral models to answer to the same 4 prompt examples, highlighted in gray in Table I:

- Brainstorming (generate a list of functional and non-functional requirements)
- Diagram generation: generate the domain model diagram (entities and relationships)
- Simulation: ask to simulate an API endpoint
- Role Playing estimation: ask the user to estimate the effort in hours of 3 use cases and give feedback.

The examples cover 3 different KAs: Software Requirements, Design, Testing and Maintenance. For each prompt we generated responses from 3 different LLMs, using the API provided by MistralAI for Mixtral8x7b (medium), the API provided by Together.AI for NousHemes/LLaMA2-70b, and the API provided by OpenAI for GPT 3.5. We chose the 3.5 version of GPT due to its free cost (so we can compare it with the free and open source alternatives, instead of using GPT4).

The system prompt for the Open Source models was: "You are a helpful software engineering assistant".

We ran the queries in a custom web application using LLMs APIs, Python and Streamlit [27] (see Fig. 4). We also developed a custom browser extension to save the results of the conversations in a database so we can export them to external systems (in our case, to Google Drive). We stored the conversations in three web anonymous Drive documents, for Mixtral ³, LLAMA ⁴, and GPT-3.5 ⁵. Then, the manuscript authors (software engineering teaching colleagues) evaluated the responses, comparing quality, usefulness and correctness across models. We also used two state of the art LLMs (GPT4 and Claude) as external evaluators or judges. To avoid biased responses (for instance, to avoid GPT4 to recognize the answers from GPT3.5) we removed any reference to the name of the LLMs in the responses to be evaluated. We uploaded three text files to each LLM-judge and used the following prompt to obtain their assessments:

You are a software engineering teacher that needs to evaluate a set of answers to software engineering questions related to the analysis, design and implementation of a Mastodon client generated by 3 Large Language Models. I have attached three text files with the answers of LLM1, LLM2 and LLM3. Read the questions (they are all the same for the three LLMs) and the answers (each file's name is linked to each LLM: llm1.txt, llm2.txt, llm3.txt). Please, evaluate the answers of each LLM regarding the quality, usefulness (how useful could the answers be for a software engineering student) and correctness. Please, the evaluation has to include a score from 1 to 5 and explain the rationale behind your answers.

VI. RESULTS

The evaluation aims to examine the capabilities of different LLMs on SE tasks through both quantitative model evaluation

³<https://tinyurl.com/mixtral-medium>

⁴<https://tinyurl.com/llama2-70b>

⁵<https://tinyurl.com/GPTv3-5>

TABLE I

A SUMMARY TABLE OF SPECIFIC EXAMPLES SHOWING HOW TO USE GENAI PROMPTS FOR TEACHING IDEAS IN EACH SWEBOK KA. THE THIRD COLUMN IS A LINK TO A CONVERSATION SAVED IN SHAREGPT.COM. IT ONLY SHOWS THE #ID OF THE CONVERSATION (USE THE LINK [HTTP://TINYURL.COM/#ID](http://tinyurl.com/#ID) TO OPEN THE LINK BY CLICKING ON IT. LINKS WITH THE SAME ID ARE PART OF A SINGLE CONVERSATION WITH THE LLM WHERE DIFFERENT PROMPTS WERE PUT TO THE TEST. HIGHLIGHTED LINES ARE PROMPTS USED IN CHATGPT, MISTRAL AND LLAMA FOR COMPARISON.

Knowledge Area		Example https://tinyurl.com/#ID
Software Requirements		
Brainstorm	Offer open questions for thinking about an issue	ypz4c53m
UML Diagrams	Generate requirements diagrams in UML	ad4seeux
Review	Give feedback to the student about the quality of their requirements	49pzep2r
Software Design		
Suggest best practices	Given a software design, offer suggestions for refactoring (e.g. for applying design patterns)	3vznttpc
Simulation	Simulate how a system would work before implementing it	4f8ae2pd
Software Construction		
Provide challenges	Provide an incomplete solution to a problem and ask/challenge the student to fill-in the gaps.	e5v7fys4
Generate new educational material	Self-taught students or learners with a desire to learn more can ask the GenAI to create new explanations for unknown or not well-known material.	y99k7px7
Software Testing		
Bootstrap testing code	Ask the students to generate testing code for a method. Then ask GenAI to do the same. Compare both solutions.	3xt832d4
Software Maintenance		
Detect bugs	Show bugged source code to students and to GenAI. Ask them to detect the bug and to explain how to solve it. Compare solutions.	cOF9ak8
Discuss source code	Discuss source code from the point of view of maintenance	2bc4ks7f
Software Configuration Management		
Automation & Scaffolding	Generate software configuration management and discuss the solution in class	48r7sv45
Software Engineering Management		
Help students to estimate	Estimating effort and cost is a difficult task for any engineer, and even more so for students with little experience.	585hvkz
Software Engineering Process		
Provide auto evaluation quizzes	Given the following text, create a quiz of 5 questions. Each question will have 4 possible answers from which only one will be correct. Explain the correct solution. Here is the text: [provide the text]	2znajakj
Help students to understand complex topics	Gradually adjusting the difficulty of the explanation "explain it to a 13 years old student"	2br9bdzm
Software Engineering Models and Methods		
Suggest an agenda of questions	GenAI can help the scrum master (a role played by a student) to prepare daily scrum meetings in advance	5n8zra6s
Software Quality		
Answer software quality related questions	E.g. "Can you give me the cyclomatic complexity value of this example() method?"	IArTqpH
Detect and refactor code smells	Explaining to the students, with specific examples, how their code smells affect software quality	mc5zxy7j
Software Engineering Professional Practice		
Stimulate discussions on related topics	E.g. suggesting open questions about legal issues in software	mrx8czap
Software Engineering Economics		
Role playing estimations	Ask about three use cases for the student to estimate in hours, then provide feedback on each estimation.	585hvkz

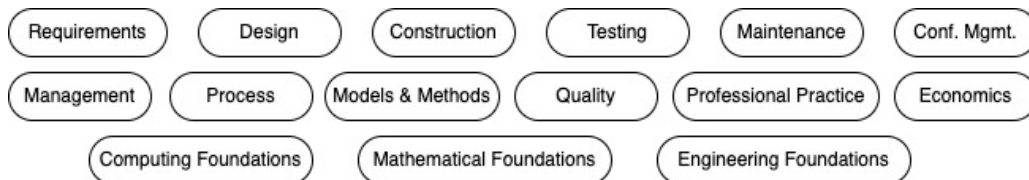


Fig. 1. SWEBOK defines 15 KAs related to software engineering. For this work, we are interested in the ones shown in figure but not in computing, mathematical and engineering foundations

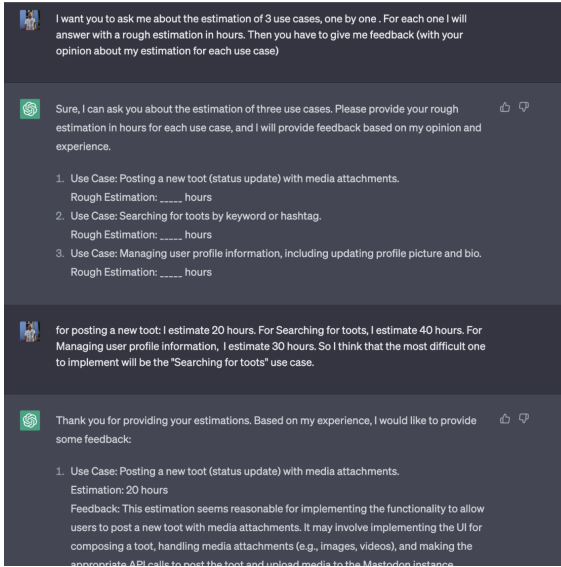


Fig. 2. GenAI can play the role of an external evaluator of students' goals, estimates and plans

TABLE II
AUTHORS EVALUATION OF THE LLM RESPONSES, COMPARING QUALITY, USEFULNESS AND CORRECTNESS ACROSS MODELS (LIKERT SCALE FROM 0 - VERY POOR - TO 5 - EXCELLENT -

Model	Quality	Usefulness	Correctness
LLaMA2-70b	2	3	2
Mixtral-8x7b	5	4	4
GPT3.5	4	4	4

and qualitative assessment. The quantitative model evaluation provides a numeric scoring of the models' outputs on key criteria relevant for SE tasks. The qualitative assessment provides a more in-depth look at differences between the LLMs' abilities to elicit use case requirements, role play the estimation of tasks, simulate an API endpoint and generate a Domain Model diagram. When combined, the quantitative and qualitative assessments provide a multifaceted understanding of the models' capabilities.

A. Model evaluation

Table II presents the authors' (human) evaluation of the quality, usefulness, and correctness of responses from different LLMs on a 5-point Likert scale. Tables III and IV show the evaluation provided by the LLM-judges.

Regarding the human evaluation, obtained by consensus after discussion, the GPT-3.5 model received high scores of 4 out of 5 on all three metrics. Mixtral-8x7b performed even better on quality with a perfect score of 5, and scored 4 on usefulness and correctness. In contrast, LLaMA2-70b received lower scores, with a 2 out of 5 on both quality and correctness, and a 3 on usefulness. Overall, the results show GPT-3.5 and Mixtral-8x7b generating higher quality, more useful, and more correct responses compared to LLaMA2-70b.

TABLE III
GPT4 EVALUATION OF THE LLM RESPONSES, COMPARING QUALITY, USEFULNESS AND CORRECTNESS ACROSS MODELS (LIKERT SCALE FROM 0 - VERY POOR - TO 5 - EXCELLENT -

Model	Quality	Usefulness	Correctness
LLaMA2-70b	4	4	4
Mixtral-8x7b	5	5	5
GPT3.5	4	4	4

TABLE IV
CLAUDE EVALUATION OF THE LLM RESPONSES, COMPARING QUALITY, USEFULNESS AND CORRECTNESS ACROSS MODELS (LIKERT SCALE FROM 0 - VERY POOR - TO 5 - EXCELLENT -

Model	Quality	Usefulness	Correctness
LLaMA2-70b	3	4	4
Mixtral-8x7b	4	5	4
GPT3.5	5	5	5

Authors' opinions are aligned with GPT4's evaluation (Table III). GPT4 rationale was the following (the exact names of each LLM have been added by the authors, in parenthesis; as explained the LLM-judges did a blind evaluation):

"LLM1 (LLaMA2-70b), LLM2 (Mixtral), and LLM3 (GPT3.5) provided high-quality, useful, and technically correct answers to the software engineering questions posed. LLM2 (Mixtral) stands out slightly in terms of quality and depth, particularly in the API draft and the detailed feedback on estimations. All models would serve as valuable educational tools, offering students practical examples of how to approach software engineering tasks."

Regarding Claude's assessment in Table IV, the judge assigned high grades to both Mixtral and GPT-3.5. However, Claude preferred the answers from GPT-3.5 over Mixtral's.

Claude's rationale for its answer was the following:

"LLM2 (Mixtral) and LLM3 (GPT3.5) provided the most comprehensive, technically sound, and useful examples for a software engineering student. Their understanding of software requirements, API design, and system modeling indicates strong technical knowledge. LLM1's (LLaMA2-70b) examples, while decent, lacked some of the rigor and polish of the other two. Overall, LLM2 and LLM3 demonstrate the most mastery."

B. Qualitative assessment

We will split our assessment in 4 parts, one for each prompt:

Models' ability to elicit use case requirements

ChatGPT provided the most comprehensive list of use cases covering major functionalities. It has detailed and appropriately named use cases like "Posting Toots" or "Timeline Viewing" and explained purpose and expected behavior for some use cases. This shows an understanding of identifying core software requirements and articulating use cases.

Mixtral also produced a reasonably thorough list of use cases with names that clearly explain intended functions. It has also grouped related cases (e.g. User Profile Management).

Fig. 3. Mixtral was the only LLM capable of generating a proper Domain Model diagram in PlantUML. GPT35 failed to generate a syntactically correct diagram so the generation failed. LLaMA2 attempt was very poor - although the diagram was syntactically correct.

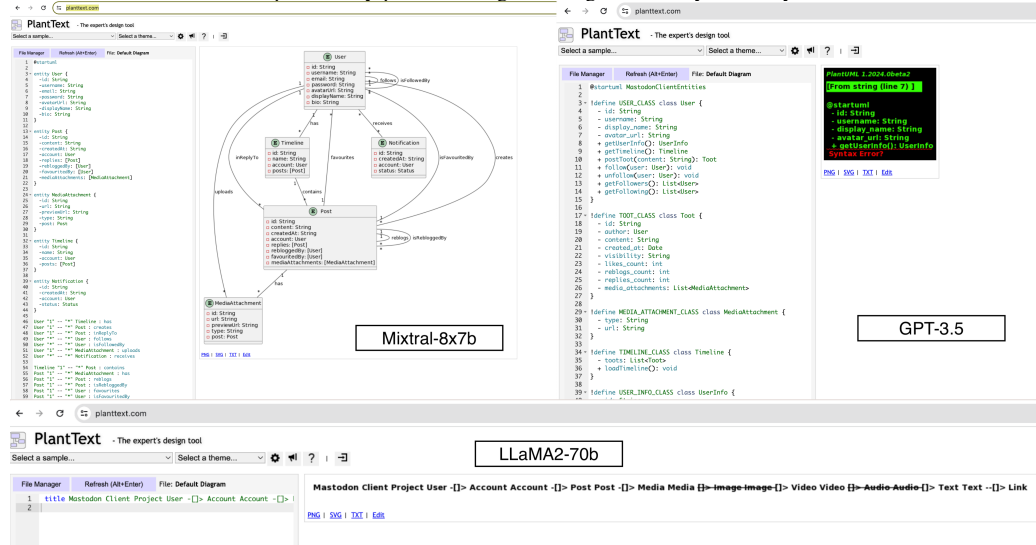
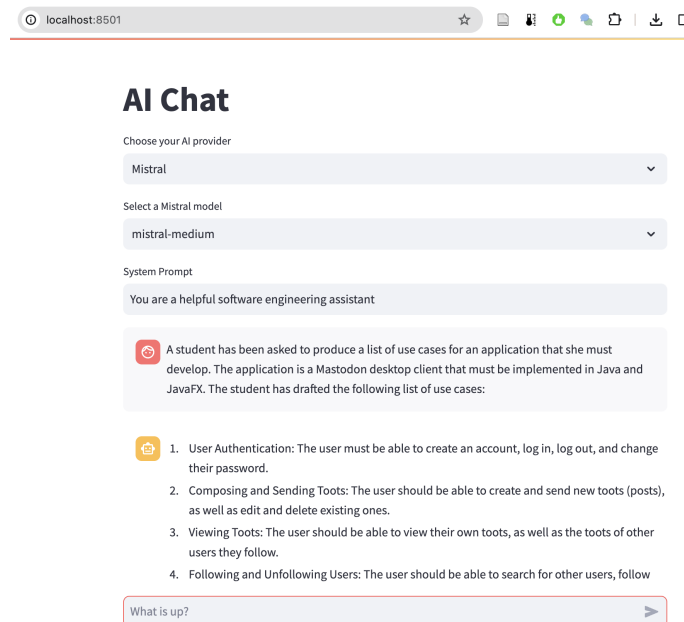


Fig. 4. We implemented a custom web interface using Streamlit for development and testing of Open Source LLMs. The conversations can be saved using a custom browser extension developed for this article.



While not as extensive as ChatGPT's, Mixtral generates a solid set of baseline use cases.

LLaMA2 starts with a very basic list of use cases that lacks the breadth or description in the other LLMs. LLaMA2 clearly has the right methodology but weaker content generation than ChatGPT or Mixtral. Those two models demonstrate greater mastery of eliciting use case requirements.

Models ability to role play the estimation of tasks ChatGPT makes a reasonable attempt at asking the student to estimate 3 use cases and provide feedback. However, provided feedback is very generic - simply stating the estimate seems reasonable without specifics and ends after just the first role

turn (even if we asked to play the role-playing game with 3 uses cases).

Mixtral has a good realistic and helpful approach to estimations. It provides context and considerations to factor in for each use case. However, it fails to play the role-playing turn based schema (it asks and answers it's own questions, simulating feedback)

LLaMA2 completely hallucinates the use cases and does not undertand the role playing game, either (it both asks and answers its own questions).

Models' ability to simulate an API endpoint. ChatGPT provides a reasonable draft GET endpoint for the Timeline

view, including expected parameters like post IDs and pagination. It also outlines request structure, response payload and codes, covering the key aspects required in an API specification, showing competence.

Mixtral generates a GET endpoint for Timeline view with an example payload. It also calls out the need to adapt to project's specific requirements and notes the lack of error handling in the draft. Mixtral also defines an appropriately scoped endpoint while acknowledging further enhancements needed.

LLaMA2, gives very basic CRUD endpoints for Timeline without including any payload structure or examples. Its response is a very basic one compared to the other two models.

Thus, ChatGPT and Mixtral produce more realistic draft endpoints with details compared to LLaMA2's cursory CRUD resources. Mixtral's self-awareness of the draft limitations is also noteworthy. Overall, ChatGPT and Mixtral better demonstrate the knowledge and communication for simulating API specification tasks.

Models' ability to generate a Domain Model diagram.

Mixtral is the only model capable of providing a complete view of a UML diagram without syntax errors in the diagram code. The domain model includes various entities such as User, Post, MediaAttachment, Timeline, and Notification with attributes and relationships detailed, like "Users can follow each other", "Posts can contain MediaAttachments", "Posts can be reblogged or favorited by Users".

ChatGPT's attempt produces an incorrect PlantUML diagram that cannot be rendered. The diagram tries to include class definitions for User, Toots (messages), MediaAttachment, and Timeline, with relationships between them. But as indicated, the output cannot even be rendered.

LLaMA's attempt is a very poor solution. It can be rendered but its quality and semantics are close to zero.

VII. DISCUSSION

Overall, ChatGPT and Mixtral produced higher quality, more useful, and more correct responses compared to LLaMA2. ChatGPT received scores of 4 out of 5 on all metrics while Mixtral received scores of 5, 4, and 4. In contrast, LLaMA2's scores were just 2, 3, and 2. ChatGPT and Mixtral consistently outperformed LLaMA2 across the evaluation prompts, demonstrating stronger capabilities for SE education tasks. But no model perfectly executed certain interactions like role playing. Although it could be due to a lack of proper prompts. The results of this analysis highlight several key implications for integrating open source LLMs like LLaMA and Mistral into software engineering classrooms:

- Open source LLMs can help students overcome initial barriers when faced with open-ended tasks like requirements gathering or estimations. The brainstorming and role playing capabilities of models like ChatGPT and Mistral demonstrate their potential to provide scaffolds to support student learning.
- Transparency of open source LLMs allows for customization and alignment with curriculum goals. Unlike black-box proprietary models, open source LLMs can be

inspected, adapted, and tuned to suit specific educational needs. This is especially relevant in specialized domains like SE.

- Running costs can be lower with local deployment options for open source models, making them accessible to more institutions. Smaller schools may lack resources for expensive proprietary services.

Although there is potential for improving software engineering education with open source LLMs, it is still important to integrate them responsibly. To ensure that there is no over-reliance on model outputs, educators must oversee and verify the results in order to look for any potential errors. It might be necessary to realign curricula and assessments in order to use LLMs to enhance rather than substitute human teaching and learning.

Infrastructure requirements like GPUs for running certain models can also incur high upfront costs for institutions. Rather than acquiring high-end GPU hardware, an alternative is to utilize inference providers to rent running time for LLMs. For this work, we used the together.ai API to run inferences on LLaMA2-70b, since we lacked local GPUs. Pricing varies by provider; at together.ai LLaMA2 costs \$0.9 per million tokens.

We elected to use the Mistral API for a clear cost structure, with Mistral-medium priced at €2.5 per million input tokens and €7.5 per million outputs. In our case, cloud-based inference enabled flexible LLM experimentation without large upfront infrastructure investments. Nevertheless, as the models advance, open source LLMs show promise as an addition to revolutionize personalized, interactive education at scale when utilized sparingly as opposed to viewing outputs as gospel. All in all, open source LLMs present valuable opportunities but require prudent integration and moderation to address ongoing challenges around hallucination and misinformation.

While LLMs offer promising opportunities, it is essential to acknowledge potential risks associated with their use in SE education. One major concern is **hallucination**, where LLMs can generate plausible-sounding but incorrect or nonsensical information, potentially misleading students. Furthermore, **biases** present in the training data can be reflected in LLM outputs, perpetuating societal biases or unfair practices. Another risk is **over-reliance on automation**, where students might become overly dependent on LLMs for generating solutions, potentially hindering the development of critical problem-solving and coding skills. Finally, **ethical considerations** arise regarding plagiarism, as students might be tempted to present LLM-generated code as their own, and intellectual property concerns might arise if LLM-generated code infringes on existing copyrights. Therefore, integrating LLMs into SE education requires careful consideration of these risks, emphasizing critical thinking, ethical practices, and the importance of validating LLM outputs.

VIII. CONCLUSIONS

In examining the role of open source LLMs in SE education, several important insights emerge:

Promising Educational Tools. Models like ChatGPT and Mistral are promising in enhancing student motivation and aiding in learning through interactive prompts. They have shown effectiveness in complex tasks such as eliciting requirements, making estimations, and drafting specifications.

Need for Oversight and Verification. While these models are useful, they are not infallible. Oversight is essential, especially in complex interactions, to ensure their outputs align with educational objectives and require validation before being integrated into the classroom.

Guidance from Established Frameworks. Frameworks like SWEBoK can inform the development of comprehensive benchmarks for prompts, specifically tailored for SE. Standardized datasets are also crucial for rigorous testing and evaluation of LLMs.

Balancing Transparency and Costs. The open-source nature of these LLMs offers opportunities for customization according to educational contexts. However, the associated infrastructure costs could be a barrier for smaller institutions. Solutions like grants or shared resources could help mitigate this.

Importance of Thoughtful Prompt Design. The effectiveness of large language models greatly depends on the design of prompts. These should be aligned with curriculum goals and if possible, follow a prompt catalog of good examples that help obtain high-quality outputs.

Studying different models in software engineering (SE) scenarios can help us learn how to better create prompts and complete tasks. As new versions of large language models (LLMs) come out, we can update our information to show their improved abilities in SE challenges. Regularly checking the latest models will show us how fast LLMs are getting better at important skills needed for SE work. By comparing different LLMs, both open source and proprietary, we can find the best ways to use these models as helpers. Open source LLMs could also make software engineering education better by offering lessons that adjust to each learner. With effective evaluation methods and teamwork, Generative AI models could change the way we teach and learn technical skills.

While our initial evaluation shows the strengths of these models in handling complex SE tasks, it is important to note that this is just the first step in a longer research path. Our future work will focus on improving the outputs of these models by fine-tuning them on SE-specific datasets (previous research [28] demonstrated LLMs trained using domain-focused corpora perform better on specialized tasks) and improving our prompt design strategies, conducting studies with our students, and including new open-source LLMs in our project.

- **Fine-tuning and Prompt Engineering:** We recognize that fine-tuning these LLMs on SE-specific datasets, along with careful prompt design, is likely to bring further improvements in their performance and educational value. Our future work will focus on developing and testing fine-tuned models tailored to specific SWEBoK KAs. We will also look more into prompt design, exploring ways to

create prompts that draw out more targeted and useful responses from the LLMs.

- **Empirical Studies with Students:** To fully understand the impact of open-source LLMs on SE education, we plan to conduct empirical studies involving students using these models as learning tools. These studies will allow us to collect data on student performance, engagement, and learning outcomes, providing concrete evidence of the educational benefits of open-source LLMs.
- **Expanding the Scope:** As the field of open-source LLMs rapidly evolves, we intend to broaden our scope to include newer models as they become available. This will provide a more comprehensive understanding of the strengths and weaknesses of different models in the context of SE education.

ACKNOWLEDGMENT

This research is supported by contract GIU21/037 under the program "Convocatoria para la Concesión de Ayudas a los Grupos de Investigación en la Universidad del País Vasco/Euskal Herriko Unibertsitatea (2021)." Additional support is provided by MCIN/AEI/10.13039/501100011033 and the European Union NextGeneration EU/PRTR under contract PID2021-125438OB-I00. Funding was also received from the University of the Basque Country (UPV/EHU) under the "University-Enterprise-Society" program (US22/17).

REFERENCES

- [1] I. Ozkaya, "Application of Large Language Models to Software Engineering Tasks: Opportunities, risks, and implications," *IEEE Software*, vol. 40, no. 3, pp. 4–8, 2023.
- [2] M. Firat, "How ChatGPT can transform autodidactic experiences and open education," *Department of Distance Education, Open Education Faculty, Anadolu Univ.*, 2023.
- [3] L. Belzner, T. Gabor, and M. Wirsing, "Large Language Model Assisted Software Engineering: Prospects, challenges, and a case study," in *Bridging the Gap Between AI and Reality*, B. Steffen, Ed. Cham: Springer Nature Switzerland, 2024, pp. 355–374.
- [4] E. Chen, R. Huang, H.-S. Chen, Y.-H. Tseng, and L.-Y. Li, "GPTutor: a ChatGPT-powered programming tool for code explanation," 2023.
- [5] J. Zhang, Y. Chen, N. Niu, and C. Liu, "A preliminary evaluation of ChatGPT in requirements information retrieval," 2023.
- [6] T.-O. Li, W. Zong, Y. Wang, H. Tian, Y. Wang, S.-C. Cheung, and J. Kramer, "Finding failure-inducing test cases with chatgpt," 2023.
- [7] J. Cao, M. Li, M. Wen, and S. chi Cheung, "A study on prompt design, advantages and limitations of chatgpt for deep learning program repair," 2023.
- [8] A. Cheshkov, P. Zadorozhny, and R. Levichev, "Evaluation of chatgpt model for vulnerability detection," 2023.
- [9] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with chatgpt," 2023.
- [10] S. Kang, J. Yoon, and S. Yoo, "Large language models are few-shot testers: Exploring llm-based general bug reproduction," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23. IEEE Press, 2023, p. 2312–2323. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00194>
- [11] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large language models for software engineering: Survey and open problems," 2023.
- [12] S. Ouhbi and N. Pombo, "Software engineering education: Challenges and perspectives," in *2020 IEEE Global Engineering Education Conference (EDUCON)*. IEEE, 2020, pp. 202–209.
- [13] I. Alves and C. Rocha, "Qualifying software engineers undergraduates in devops - challenges of introducing technical and non-technical concepts in a project-oriented course," *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, pp. 144–153, 2021.
- [14] S. Murthy, "Difficulty of Software Engineering and Ways to Overcome Common Challenges," Feb. 2023. [Online]. Available: <https://tinyurl.com/wguedublog>
- [15] O. Shabalina, N. Sadovnikova, and A. Kravets, "Methodology of teaching software engineering: Game-based learning cycle," in *2013 3rd Eastern European Regional Conference on the Engineering of Computer Based Systems*. IEEE, 2013, pp. 113–119.
- [16] N. Battaglia, C. Neil, and M. De Vincenzi, "Software engineering competence-based learning in collaborative virtual environments," in *2021 IEEE World Conference on Engineering Education (EDUNINE)*, 2021, pp. 1–5.
- [17] N. Ngadiman, S. Sulaiman, N. Idris, M. R. Samingan, and H. Mohamed, "Checklist approach for the development of educational applications by novice software developers," *IEEE Access*, vol. 11, pp. 900–918, 2023.
- [18] O. Shata, "Teaching software engineering: Problems and suggestions," *International Journal of Computer Science Issues (IJCSI)*, vol. 8, no. 1, p. 232, 2011.
- [19] IEEE Computer Society, *SWEBOK: Guide to the Software Engineering Body of Knowledge*. Piscataway, NJ: IEEE Press, 2014.
- [20] J. G. Meyer, R. J. Urbanowicz, P. C. Martin, K. O'Connor, R. Li, P.-C. Peng, T. J. Bright, N. Tatonetti, K. J. Won, G. Gonzalez-Hernandez *et al.*, "Chatgpt and Large Language Models in academia: opportunities and challenges," *BioData Mining*, vol. 16, no. 1, p. 20, 2023.
- [21] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, L. Gutierrez, T. F. Tan, and D. S. W. Ting, "Large language models in medicine," *Nature medicine*, vol. 29, no. 8, pp. 1930–1940, 2023.
- [22] HuggingFace. (2023) Quantization. [Online]. Available: https://huggingface.co/docs/optimum/concept_guides/quantization
- [23] Y. Fan, F. Jiang, P. Li, and H. Li, "Grammargpt: Exploring open-source llms for native chinese grammatical error correction with supervised fine-tuning," in *CCF International Conference on Natural Language Processing and Chinese Computing*. Springer, 2023, pp. 69–80.
- [24] M. Müller, C. Schindler, and W. Slany, "Engaging students in open source: Establishing foss development at a university," 2019.
- [25] R. Plant, V. Giuffrida, and D. Gkatzia, "You are what you write: Preserving privacy in the era of large language models," *arXiv preprint arXiv:2204.09391*, 2022.
- [26] A. Khelifi, M. Talib, M. Farouk, and H. Hamam, "Developing an initial open-source platform for the higher education sector—a case study: Alhosn university," *IEEE transactions on learning technologies*, vol. 2, no. 3, pp. 239–248, 2009.
- [27] Streamlit • a faster way to build and share data apps. [Online]. Available: <https://streamlit.io/>
- [28] B. Bhattacharjee, A. Trivedi, M. Muraoka, M. Ramasubramanian, T. Udagawa, I. Gurung, R. Zhang, B. Dandala, R. Ramachandran, M. Maskey, K. Bugbee, M. Little, E. Fancher, L. Sanders, S. Costes, S. Blanco-Cuaresma, K. Lockhart, T. Allen, F. Grezes, M. Ansdell, A. Accomazzi, Y. El-Kurdi, D. Wertheimer, B. Pfitzmann, C. B. Ramis, M. Dolfi, R. T. de Lima, P. Vagenas, S. K. Mulkavilli, P. Staar, S. Vahidinia, R. McGranaghan, A. Mehrabian, and T. Lee, "Indus: Effective and efficient language models for scientific applications," 2024.